

LASAL

LASAL Motion Control API

기능 설명서

LasalMotionControlLib 공개 C# API 의
기능, 호출 인자 UNIT 과 반환값

문서	API 기능 설명서
적용 API	LasalMotionControlLib 0.9.1-preview
환경	Windows, .NET Framework 4.8
발행	2026-08-12
문서 버전	2.3-candidate

API 기능과 호출에 필요한 값만 간단히 설명합니다.

개정 이력

문서 버전	날짜	내용
1.0	2026-07-15	최초 작성
1.1	2026-07-16	공개 API 레퍼런스 작성
1.2	2026-07-16	API 기능, 인자 UNIT 과 반환값 중심으로 간소화
1.3	2026-07-16	preview 안전 경고, 응답 판정과 4 축 group 제한 보완
1.4	2026-07-16	group position read 계약 불일치와 static identity 제한 명시
1.5	2026-07-22	read-only Admin, typed drive status, PI/Bulk facade 와 local error catalog 추가
1.6	2026-07-23	0x7D22 GroupMoveLinearRelative API, wire/state 제한과 runtime 검증 경계 추가
1.7	2026-07-29	Axis Power/Reset/Stop 과 Group Stop accepted-once stable wait, deadline evidence 추가
1.8	2026-07-29	Axis Stop/Reset process-local mutation 귀속, Reset Begin/Resume 와 WPF no-replay 정책 추가
1.9	2026-07-30	신규 wait/resume, drive error read 와 D0~D5/Recorder/Topology API 를 반영하고 current 지원 상태와 완료 판정 경계 추가
2.0-candidate	2026-07-31	Axis1-only SDO Write identity-pinned four-ticket gate, stale recovery retirement, single-instance 실행과 transactional Distribution candidate 경계 추가
2.1-candidate	2026-08-04	LMC Home current-position-zero start/outcome/retirement, DS402 Home gate 상태와 TW19/TW20 encoder maintenance 계약 추가
2.2-candidate	2026-08-11	14ccf58 exact canonical -1 bounded fresh-TCP reconnect, complete local cleanup, startup identity 와 PC 검증 경계 추가
2.3-candidate	2026-08-12	cbf2548 actual EXE X 종료/재실행과 binary identity gate, 3c63dea 13-role active Python dependency closure, d4204b4 exact Gate D PC/static snapshot 승인, RPC_INIT_FRESH_TCP_ONCE_V2 bounded pre-response transport recovery 와 canonical tracked release-input 경계 추가

이 문서는 LasalMotionControlLib.dll 의 API 기능과 호출 인자, UNIT, 반환값을 설명하는 빠른 참조다. 모든 공개 diagnostic event/property 를 열거한 완전한 API reference 는 아니다.

Preview/안전 경고: 0.9.1-preview 는 production 승인본이 아니다. 2026-07-30 historical checkpoint 의 SourceOnly/full static 과 fresh IDE Rebuild/Link 는 PASS 했다. d4204b4 는 clean tracked Classes.lcb exact physical snapshot(8,549,773 bytes, SHA-256 24402BFA76F1989319381388D4354E1528052078BA08504CC5C967A6DE1AA861)의 TerminalWakeBrokerCandidate 만 PC/static ProductionApproved=true, NeedsRebaseline=false 로 승인했다. Verifier selftest 는 PS5.1/PS7 각각 296/296, Verify-LasalContract.ps1 -SourceOnly 는 두 host 에서 Phase5TransportClean/IntegratedReadOwnerDormant checkpoint 로 PASS 했다. Main worktree 의 사용자 Classes.lcb(8,549,773 bytes, SHA-256 D4C1FF4650499777A17854DA638269543938532520F0C5D178D61FF13BAA0C36)는 exact identity drift 로 계속 reject 된다. 이 ratchet 뒤 clean full Distribution 은 아직 실행하지 않았으므로 current schema 3 candidate, actual EXE, manifest 와 publish 는 생성하거나 검증하지 않았다. Current reconnect source 와 일치하는 PLC image 의 LASAL build/download 는 2026-08-12 15:58 에 완료했다. 같은 창 Close -> Connect live reconnect PASS 와 Motion/Power/SDO Write runtime proof 는 아직 없다. Image transfer 를 runtime proof 로 해석하지 않는다. LMC_Response.IsSuccess 는 frame 과 command 수락 결과이지 motion, power 전이, Stop 완료가 아니다. typed status/position 을 polling 한다. CloseConnection, Dispose, timeout 과 cancellation 은 PLC motion Stop 이나 safe-stop 을 보내지 않는다. Motion/Group 전체 25-command PLC matrix 는 아직 완료되지 않았다. Admin/Group relative/ Stop/PowerOff 와 D1 Catalog/PI, D2 Bulk, D5 axis 1.4 의 기존 happy path 를 D1/D2/D5 fault/soak 또는 D3/D4 runtime/reconnect-adopt 완료로 확대 해석하지 않는다. 실제 장비에서는 E-stop 과 HW/SW limit, UNIT, Home/Reference, 이동 범위와 explicit safe-stop 을 장비별로 별도 승인한다. 배포 DLL 은 strong-name 및 AuthentiCode 서명이 없으므로 승인된 package manifest 와 SHA-256 으로 출처를 확인한다. Admin 0x7D00/10/20/22 는 source/static 과 current LASAL IDE build 까지 완료했지만 PLC download 와 실물 parameter 값/UNIT/relative motion 은 아직 검증하지 않았다.

canonical release-input 상태: 이 Markdown 은 2026-08-12 current development source 용 2.3-candidate 원본이다. canonical Distribution DOCX/PDF 는 이 원본에서 생성하고 독립 검토한 2.3-candidate 와 동일한 tracked release-input baseline 이다. d4204b4 의 Gate D 승인은 위 exact clean tracked snapshot 의 PC/static 경계만 닫으며 main worktree identity drift 는 승인하지 않는다. 이 전환은 clean release candidate 생성을 위한 입력 승격일 뿐 production 승인, current PLC live proof 또는 full Distribution PASS 가 아니다. Clean detached checkout 의 full Distribution 과 release-scope 승인이 끝나기 전에는 canonical 2.3-candidate 를 production 배포 매뉴얼로 간주하지 않는다.

목차

개정 이력.....	2
1. 공통 사항	6
1.1 Assembly.....	6
1.2 UNIT 변환.....	6
1.3 공통 반환값.....	6
1.4 Enum 값	7
1.5 현재 지원 상태와 완료 판정.....	7
1.6 accepted-once wait/resume 공통 모델.....	8
1.7 WPF recovery identity mismatch 해소	9
2. Connection API	10
2.1 LMCCConnection.....	10
2.2 LMCCConnectionOptions.....	10
2.3 RpclnitConnection.....	10
2.4 CloseConnection.....	12
2.5 Safety transport preemption.....	13
3. Single Axis API.....	14
3.1 LMCSingleAxis 생성.....	14
3.2 PowerOn	14
3.3 PowerOff	15
3.4 Reset.....	16
3.5 Stop	17
3.6 ReadStatus	18
3.7 GetActualPosition	19
3.8 MoveAbsoluteEx	19
3.9 MoveRelativeEx	20
3.10 MoveVelocityEx.....	21
3.11 Drive operation mode 와 composite status.....	21
3.12 Move 완료와 restart recovery 경계	22
4. Group API	22
4.1 LMCGroupAxis 생성.....	22
4.2 GetGroupMembersInfo	23
4.3 GroupPowerOn.....	23
4.4 GroupPowerOff	24
4.5 GroupEnable.....	24
4.6 GroupDisable	25
4.7 GroupReset.....	26
4.8 GroupStop.....	28
4.9 GroupReadStatus.....	29
4.10 GroupReadActualPosition	29
4.11 SetKinTransformCartesian4Axis.....	30
4.12 MoveLinearAbsoluteEx	30
4.13 MoveLinearRelativeEx.....	31
4.14 LMCGroupMotionOptions	32
5. Return Type.....	33
5.1 LMCReadStatusResult	33
5.2 LMCReadActualPositionResult	33

5.3 LMCGroupReadStatusResult.....	34
5.4 LMCGroupReadActualPositionResult	34
5.5 LMCGroupMembersInfoResult	35
5.6 LMCAdminResponse	35
6. Admin 과 Diagnostics facade	36
6.1 Admin capability 와 semantic parameter read.....	36
6.2 PI alias 와 Bulk builder/reader	37
6.3 Project-local error catalog	37
6.4 Diagnostics D0 capability 와 공통 수명.....	37
6.5 Diagnostics D1 Catalog, Health 와 PI Read.....	38
6.6 Diagnostics D2 Bulk Read.....	39
6.7 Diagnostics D3/D4 Recorder.....	39
6.8 Diagnostics D5 SDO Read 와 operation ticket	40
6.9 Static Topology 와 Dynamic Node/I/O.....	40
6.10 Mutation API 정책	41
6.11 Request/result 와 provenance 확인.....	41
6.12 LMC Home, DS402 Home 과 Encoder Maintenance.....	42

1. 공통 사항

1.1 Assembly

항목	값
DLL	LasalMotionControlLib.dll
Namespace	LasalMotionControlLib
Framework	.NET Framework 4.8
API version	0.9.1-preview

1.2 UNIT 변환

API 의 motion 인자는 `Int32` DINT 다. API 는 UNIT 을 자동으로 곱하거나 나누지 않는다.

송신 DINT = 물리값 x PLC application UNIT
수신 물리값 = 수신 DINT / PLC application UNIT
Jerk 송신 DINT = (물리 jerk / 1000) x PLC application UNIT

값	API 인자 UNIT	설명
Position / Distance	PLC application UNIT DINT	예: mm UNIT 이 10000 이면 1 mm 는 10000
Velocity	PLC application UNIT/s DINT	예: 1 mm/s 는 UNIT 10000 기준 10000
Acceleration / Deceleration	PLC application UNIT/s ² DINT	PLC 에 설정된 application UNIT 사용
Jerk	PLC application UNIT/s ³ /1000 DINT	1000 mm/s ³ 는 입력값 1 에 UNIT 을 곱함
Actual position	PLC application UNIT DINT	반환값을 UNIT 으로 나누어 물리값 계산

주요 상수는 다음과 같다. 실제 PLC 에 설정된 UNIT 이 다르면 그 값을 사용한다.

Constant	값	의미
LMC_Units.MM	10000	Millimeter
LMC_Units.M	10000000	Meter
LMC_Units.DEG	10000	Degree
LMC_Units.MMPSEC	10000	Millimeter/second
LMC_Units.RPM	1000	Revolution/minute

1.3 공통 반환값

제어와 motion 명령은 `LMC_Response` 를 반환한다.

Property	Type	설명
IsSuccess	bool	frame 과 command 결과 성공 여부; motion/power/stop 완료를 뜻하지 않음
Raw / Payload	byte[]	방어 복사된 원본 frame/payload
HeaderStatus	ushort	response envelope 상태
PayloadLength	ushort	header 에 선언된 payload 길이
HeaderReserved	uint	header reserved 값
IsFrameValid	bool	header 와 command 별 payload shape 검증 결과
HasCommandResult	bool	command status/error field 존재 여부
CommandStatus	ushort	command/function status
Status	ushort	command result 가 있으면 CommandStatus, 없으면 HeaderStatus
ErrorId	short	반환된 error ID, 정상은 0

비동기 메서드는 동일한 결과를 Task<LMC_Response>로 반환한다.

1.4 Enum 값

Enum	사용 값	설명
LMC_DIRECTION	Shortest	Absolute / Relative motion
LMC_DIRECTION	Positive, Negative	Velocity motion 방향
LMC_COORD_SYSTEM	None, Acs, Mcs, Pcs	Group 좌표계
LMC_BUFFER_MODE	Aborting, Buffered	현재 배포 PLC 에서 사용하는 buffer mode
LMC_GROUP_TRANSITION_MODE	ExactStop, ContinuousDirect	현재 배포 PLC 에서 사용하는 transition mode

1.5 현재 지원 상태와 완료 판정

아래의 source-active 는 C# API 와 LASAL route 가 current source 에 있다는 뜻이다. production 승인이나 전체 실제 장비 검증을 뜻하지 않는다.

영역	Current 상태	사용 범위와 남은 확인
Connection / Axis / Group core	source-active, PLC 검증 부분	대표 lookup, power, move, stop/read 경로가 있으나 전체 command/fault/race matrix 는 미완료
Axis/Group accepted-once wait/resume	PC public API, 기존 opcode 재사용	ACK 뒤 status-only polling 과 no-replay recovery 를 제공하며 current same-hash/실장비 전체 회귀가 필요

영역	Current 상태	사용 범위와 남은 확인
Admin 0x7D00/10/20/22	source-active, static/IDE build 검증	semantic read와 Group relative move 의 current PLC download/실물 UNIT 검증 필요
LMC Home 0x7D13/18/19	source-active, Admin bit 4 ON	CurrentPositionZero 는 no-motion/no-switch 이며 Axis2 raw 1-count false -7 수정 뒤 C78 Rebuild/Download 와 한 축 terminal runtime proof 필요
DS402 Home 0x7D15/16/17	method 37 source 구현, Admin bit 6/gate OFF	current runtime 실행 금지; source 존재와 terminal proof 를 구분
TW[20]/TW[19] 0x7E53/54/55	fixed 0x20FC:2/:1 <- UInt16 1, source bits 18/19 ON	protocol terminal 과 선택 drive 의 실제 물리 효과를 별도로 검증
D1/D2/Recorder/D5 Read	C# facade 와 PLC route 가 단계별 구현	capability 를 먼저 읽고 fault, reconnect, soak 와 physical readback evidence 를 추가해야 함
Static EtherCAT topology 0x7E11/12	configured 7-entry inventory	topology qualifier durable report 와 current PLC identity 확인 필요
Node Health / Digital I/O 0x7E13/22/23	0x7E13/22 LASAL source 구현, capability OFF; 0x7E23 없음	current PLC download/runtime proof 전에는 정상 UI/API 에서 호출하지 않음
SDO Write	축 1 UI[24] exact target 만 source/IDE build 승인	current PLC download 와 bit 9 확인 후 제한 시험; 실기 mutation evidence 미완료
PI Write / Recorder Double	gate 또는 allowlist OFF	별도 승인과 실제 장비 mutation evidence 전까지 사용하지 않음

완료 판정은 ACK -> typed status polling -> stable sample -> final readback 순서로 한다.

timeout/cancellation/connection close 는 장비 정지나 command 취소를 자동 보장하지 않는다.

1.6 accepted-once wait/resume 공통 모델

Power, Stop, Reset 과 일부 Group command 는 ACK 와 완료 상태를 분리하고, 한 번 수락된 command 를 다시 보내지 않은 채 status-only polling 을 재개하는 API 를 제공한다.

구성	기본값 / 의미
...WaitOptions.TotalTimeoutMilliseconds	5000 ms, 허용 1~600000 ms
...WaitOptions.PollIntervalMilliseconds	50 ms
...WaitOptions.StableSampleCount	3 회, 허용 1~100 회
...SubmissionOutcome	NotAttempted, Rejected, OutcomeUncertain, Accepted
...WaitContinuation	같은 connection/session/object 에 묶인 pending ACK 와 polling evidence
...WaitResult	final status, continuation 과 submission/polling evidence
Pending...WaitContinuation	현재 handle 에서 아직 완료되지 않은 latest accepted continuation;

구성	기본값 / 의미
	없으면 null

```

LMCAxisStopWaitContinuation pending =
    await axis.BeginStopWaitForStableStandstillAsync(
        deceleration,
        jerk,
        cancellationToken);

LMCAxisStopWaitResult stopped =
    await axis.ResumeStopWaitForStableStandstillAsync(
        pending,
        cancellationToken);

```

Continuation 은 server-side idempotency key 나 영구 token 이 아니라 현재 process 의 connection/session-bound 객체다. exact pending continuation 으로 **Resume...**을 호출해야 하며, 새 **Begin...** 호출을 blind retry 로 사용하지 않는다. pre-write cancellation 은 보통 zero-wire 지만, **CommandMayHaveBeenSent=true** 또는 **OutcomeUncertain** 이면 자동 replay 하지 않는다. **TransportInvalidatedAtDeadline=true** 이면 reconnect 와 object lookup 을 다시 수행한다. pending property 는 durable storage 가 아니므로 process restart 뒤 복원되지 않는다. restart 뒤에는 motion command 를 재전송하지 말고 read-only **WaitFor...** helper 또는 explicit Stop/Power Off 를 사용해 현재 상태를 다시 확인한다. 단, API 가 별도 durable recovery record/attach 계약을 명시한 Axis/Group operation 은 old continuation 을 복원하는 대신 exact endpoint/PLC/target identity 를 재검증해 새 session-bound status-only continuation 을 만든다. 이 경로도 원 command 를 replay 하지 않는다.

1.7 WPF recovery identity mismatch 해소

개발 WPF 가 저장된 recovery record 와 현재 PLC 의 **BootId** 또는 **MapRevision** 불일치를 표시하면 Motion, Power, Group control, D5/SDO Write 와 qualification 은 모두 차단된다. 이 상태에서 현재 PLC 의 status 가 정상이라는 이유로 과거 command 결과를 성공으로 판정하면 안 된다. Group Reset record 는 **DiagnosticsBuild** 도 exact identity 에 포함하므로 Build-only mismatch 도 같은 read-only quarantine 대상이다.

해당 record 가 현재 endpoint 의 이전 PLC identity 에 속하고 운영자가 장비와 드라이브의 물리 상태를 독립 확인한 경우에만 화면의 stale recovery 목록을 검토한다. 확인 checkbox 와 경고창을 승인한 뒤 **Archive and Retire Stale Recovery** 를 실행한다. 이 작업은 PLC command 를 보내지 않고 원 journal 과 SHA-256 을 immutable retirement ledger 에 보존한 뒤 exact journal CAS 로만 record 를 retire 한다. 혼합 record 에서는 현재 endpoint 의 stale subset 만 **RETIRE STALE** 로 폐기하고 exact-current 와 다른 endpoint record 는 각각 **KEEP EXACT CURRENT**, **KEEP OTHER ENDPOINT** 로 보존한다. 완료되면 기존 연결이 닫히고 앱이 종료된다. 새 process 에서 다시 연결해 남은 exact status-only recovery 를 완료해야 Motion/Power/approved SDO Write admission 이 재평가된다. ledger 또는 journal fault, endpoint 불일치, 실행 중 operation 이 있으면 절차는 fail-closed 다. retirement 는 Build-bearing Group Reset record 에 대해서는 current **DiagnosticsBuild** 까지 confirmation 전후 두 번 다시 읽고, 기존 Build-less record 는 이전 BootId/MapRevision 계약을 유지한다.

2. Connection API

2.1 LMConnection

Connection 객체를 생성한다. 생성만으로 PLC 에 연결되지는 않는다.

```
public LMConnection()
public LMConnection(LMConnectionOptions options)
```

Parameter	Type	UNIT	설명
options	LMConnectionOptions	-	Connection timeout 설정

Return	설명
LMConnection	생성된 connection 객체

2.2 LMConnectionOptions

Connection timeout 과 callback 검증 값을 설정한다.

Property	Type	UNIT	Default
ConnectTimeoutMilliseconds	int	ms	3000
ReceiveTimeoutMilliseconds	int	ms	3000
SendTimeoutMilliseconds	int	ms	3000
CallbackThreadJoinTimeoutMilliseconds	int	ms	500
ValidateCallbackSourceAddress	bool	-	true
CallbackRegistrationMode	LMCallbackRegistrationMode	-	LegacyRaw
CallbackRequestedMaxDatagramBytes	int	bytes	512
SendPriorityCoordinator	LMSendPriorityCoordinator	-	선택적 PC-side safety send coordination

2.3 RpcInitConnection

PLC TCP 연결, RPC 초기화와 callback 등록을 수행한다.

```
public void RpcInitConnection(
    string remoteAddress,
    int remotePort,
    string localAddress)

public void RpcInitConnection(
    string remoteAddress,
    int remotePort,
```

```

    string localAddress,
    int callbackPort,
    uint eventMask)

public Task RpcInitConnectionAsync(
    string remoteAddress,
    int remotePort,
    string localAddress,
    CancellationToken cancellationToken)

public Task RpcInitConnectionAsync(
    string remoteAddress,
    int remotePort,
    string localAddress,
    int callbackPort,
    uint eventMask,
    CancellationToken cancellationToken)

```

Parameter	Type	UNIT / 범위	설명
remoteAddress	string	IPv4	PLC IP address
remotePort	int	1~65535	PLC TCP port
localAddress	string	IPv4	PLC 와 연결되는 PC NIC address
callbackPort	int	0~65535	PC UDP callback port, 기본값 5003
eventMask	uint	Bit mask	Callback event mask, 기본값 0xFFFFFFFF
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
void	동기 초기화 완료
Task	비동기 초기화 작업

library default `LegacyRaw` 는 callback registration 12-byte request/4-byte ACK 를 사용한다. 명시적 `Version2WakeHint` 는 32/20 과 strict 52-byte typed wake 를 사용한다. 이 mode 에서 outer `Status=1`, `HeaderReserved=0`, payload 4, command `Status=1`, `ErrorId=-1` 인 exact canonical init failure 만 20 ms 뒤 같은 TCP socket 에서 `0x8080` 을 한 번 더 보낸다.

`LastRpcSessionInitializationEvidence` 는 `Outcome`, `AttemptCount`, `CanonicalRetryUsed`, 첫/마지막 response 를 cleanup 뒤에도 보존한다. legacy, `ErrorId=0`/다른 error, nonzero reserved 와 malformed response 는 SDK retry 가 없다.

SDK 자체는 새 TCP connection 을 자동으로 만들지 않는다. 개발 WPF 의 current `RPC_INIT_FRESH_TCP_ONCE_V2` 는 첫 candidate 예만 fresh-TCP budget 을 준다. (A) exact canonical -1 두 개로 `AttemptCount=2`, `CanonicalRetryUsed=true` 가 된 persistent same-socket failure 는 100 ms 뒤, (B) 실제 `0x8080` request 가 시작된 `AttemptCount=1`, response 없음과 direct `EndOfStreamException/SocketException/TimeoutException`, 또는 그중 하나를 `InnerException` chain 에 가진 `IOException` 인 pre-response transport failure 는 1000 ms 뒤 새 `LMCConnection/TCP` 하나를 연다.

두 번째 candidate failure 는 terminal 이다. Connect-before-init 은 `AttemptCount=0` 이며 retry 가 없다. cancellation, `ObjectDisposedException`, `InvalidDataException`(허용형 `InnerException` 이 있어도 포함), malformed response, valid non--1 response, response 이후와 callback-stage failure 에도 fresh-TCP retry 가 없다. One UI operation is bounded

to TCP 2 and 0x8080 4 requests. `0x405C` 는 init 성공 뒤에만 나가며 정상 registration ACK 까지 받아야 Connect 가 성공한다. Historical `14ccf58` V1 은 persistent--1 만 허용했던 과거 policy 다.

100/1000 ms 는 PC bounded backoff 이지 PLC readiness 증거가 아니다. canonical wire -1 만으로 내부 disarm -8/-9 와 다른 lifecycle/ownership rejection 을 구분할 수 없다. PC fake-RPC 와 loopback test 는 PLC runtime proof 가 아니다. PC cleanup 은 PLC disarm 성공을 뜻하지 않으며 private PLC state 를 force-clear 하지 않는다. Same-window Close -> Connect live reconnect is not verified.

2.4 CloseConnection

RPC connection 과 local TCP/UDP resource 를 닫는다.

```
public void CloseConnection()
public Task CloseConnectionAsync(CancellationTokent cancellationTokent)
public void Dispose()
```

Parameter	Type	UNIT	설명
cancellationTokent	CancellationTokent	-	비동기 호출 취소 token

Return	설명
void	동기 종료 완료
Task	비동기 종료 작업

`CloseConnection`, `CloseConnectionAsync` 와 `Dispose` 는 Axis/Group Stop 또는 Power Off 를 보내지 않는다. strict `CloseConnection[Async]` 는 초기화된 연결에서 가능한 경우 RPC close `0x405D` 를 시도한 뒤 local socket/resource 를 정리한다. `Dispose` 는 캡처한 close protocol/transport 오류를 local cleanup 경로 뒤 다시 throw 하지 않지만, failed/uninitialized candidate 에서는 `0x405D` 가 나가지 않을 수 있다. nonzero close ACK 나 close transport 오류가 있어도 local cleanup 뒤 `RpcCloseResponse` 와 `LastCloseException` 을 확인할 수 있다. strict `CloseConnection[Async]` 는 cleanup 뒤에도 오류를 호출자에게 전달한다. 장비를 안전하게 정지해야 하면 connection close 전에 use an explicit safe-stop procedure 를 적용한다.

개발 WPF 의 내부 replacement 와 창 X 는 공용 최대 2 회 `Dispose` cleanup 후 `Disconnected`, TCP/RPC/callback 정지와 endpoint null 을 모두 요구한다. X 는 이 postcondition 이 완성되지 않으면 종료를 취소한다. startup log 에는 `ReconnectPolicy=RPC_INIT_FRESH_TCP_ONCE_V2`, `SdkPath`, `SdkBuildUtc` 가 있으며 topology marker V5 는 유지된다. RPC-init evidence 에는 `CandidateOrdinal`, `FreshSessionRetryReason`, `FreshSessionRetryDelayMs`, `FreshSessionRetryFromCandidate`, `FreshSessionRetryNextCandidate`, `FreshSessionFirstFailure` 가 포함된다. Historical 검증은 SDK Debug/Release direct 1133/1133, WPF Debug/Release Rebuild PASS, 기존 full smoke 339/339, reconnect targeted 6/6 PASS 이고 독립 callback/reconnect review 는 9/9, P0/P1 없음이다. 이 수치를 V2 final count 로 소급하지 않는다. Current V2 는 Release full smoke 347/347, isolated Debug build 와 `Wpf.CallbackV2.* 17/17` 을 PASS 했다. 이는 PC fake-RPC 증거이며 PLC runtime proof 가 아니다. Targeted suite 에는 동일 `MainWindow`, 동일 fixed UDP port 에서 explicit Close 뒤 endpoint rebind/new TCP Connect 를 확인한 정상 ACK 와 exact -1 변형이 포함된다. Historical fake restart 는 같은 test process 의 새 `MainWindow` 를 사용하는 별도 회귀다.

Historical executable-gate checkpoint `cbf2548` 의 별도 actual-EXE relaunch gate 도 Debug/Release 각각 1/1 PASS 했다. Runner 가 실제 EXE PID/HWND 에 외부 `WM_SYSCOMMAND/SC_CLOSE` 를 보내 owner 의 X close 를 실행하고, `0x405D` exact -1 뒤 bounded cleanup/process exit 를 확인한다. live owner 중 contender 는 default mutex 에서 exit 2, TCP session 0 이고 owner exit 뒤 같은 exact EXE successor 가 mutex 를 재획득한다. Successor 의 첫 candidate 는 `0x8080` exact -1 두 번과 `0x405C/0x405D` 0 회, fresh candidate 는 init/registration/close 성공이다.

전체 fake-RPC session/request exact wire total 은 3/28 (13,2,13)이다.

malformed probe 는 exit 64, owned temp write 0, TCP session 0 이고 EXE/SDK DLL/optional config identity 는 시험 전후 동일하다.

이 gate 는 PC loopback process/default-mutex/local-cleanup/wire 증거다. PLC cleanup/disarm/readiness, 100/1000 ms 의 PLC 적정성, MotionLib/축 상태 또는 사용자 PLC 의 실제 종료 후 재접속을 증명하지 않는다. PC cleanup 은 PLC disarm 성공이 아니며 private PLC state 를 force-clear 하지 않는다. V2 fake-RPC test 와 15:58 image build/download 도 같은 창 Close -> Connect live reconnect PASS 를 대체하지 않는다.

배포 script 는 candidate Run copy 직후, manifest 전에 actual-EXE gate 를 실행하고 transaction 완료 전에 tested/final EXE SHA-256 equality 를 요구한다. 별도 binary-reference temp candidate(`ProjectReference=0`, config absent)는 gate 를 PASS 했고 EXE SHA-256 은 `829AC3314E1B5113696DFA06E64418A95C305035335F73DEB4404449CF910F79`, SDK SHA-256 은 `7D179781BCE9EB2FE6DB071C3D45F085A5BC127F9DBD0E15300E38A6181A7ED8` 이었다. Historical full Distribution 첫 attempt 는 gate 보다 앞선 `Verify-LasalContract.ps1:7571 $macroMatches[-1]`의 PowerShell 5.1 비호환 tooling bug 에서 중단됐고 transaction residue 는 0 이다. PLC/source/Classes/`cbf2548` blocker 가 아니다. 후속 pwsh7 focused verifier-only 는 exit 0, negative fixture 62/62 reject 와 comment-only fixture accept 를 64.3 초에 PASS 했다. Compatibility commit `ad4af91` 은 verifier 의 PS5.1 negative-index 접근만 수정했고 targeted PS5/PS7 Publish+Reserve 를 PASS 했다. 수정 뒤 PS5.1 Release `RunLasalContract/RunLasalNetworkContract` 는 해당 경계를 통과한 다음 각각 177.7 초/174.9 초에 기존 intentional `LASAL.UdpCallbackContract blocker: Classes.lcb sanctioned Gate D identity drifted` 로 exit 1 이었다.

`d4204b4` 는 clean tracked `Classes.lcb` 의 8,549,773-byte exact SHA-256 `24402BFA76F1989319381388D4354E1528052078BA08504CC5C967A6DE1AA861` TerminalWakeBrokerCandidate 만 `ProductionApproved=true, NeedsRebaseline=false` 로 승인했다. Selftest PS5.1/PS7 296/296 과 두 host 의 SourceOnly `Phase5TransportClean/IntegratedReadOwnerDormant` checkpoint 는 PASS 했다. 반면 수정하지 않은 main worktree 사용자 `Classes.lcb` SHA-256 `D4C1FF4650499777A17854DA638269543938532520F0C5D178D61FF13BAA0C36` 는 sanctioned identity drift 로 계속 reject 된다. Ratchet 뒤 clean full Distribution 은 아직 실행하지 않아 STOP 이며, current schema 3 candidate, actual EXE, manifest 와 publish PASS 가 아니다. 이 `d4204b4` checkpoint 당시 LASAL IDE build/PLC Download/runtime 은 실행하지 않았다. 이후 current reconnect source 와 일치하는 PLC image 는 2026-08-12 15:58 에 build/download 했지만, 같은 창 Close -> Connect 와 Motion/Power/SDO Write runtime 은 여전히 검증하지 않았다.

2.5 Safety transport preemption

일반 command 가 transport 를 점유하거나 결과 publication 대기 중일 때, 상위 safety owner 가 local TCP transport 를 폐기하고 새 session 에서 safety command 를 다시 소유하기 위한 PC-side API 다.

```
public LMCSafetyPreemptionAbortEvidence
    AbortTransportForSafetyPreemption()
```

```
public LMCSafetyPreemptionAbortEvidence
  AbortTransportForSafetyPreemption(
    long expectedSessionGeneration)
```

이 호출은 RPC Close, Axis Stop, Group Stop 또는 Power Off frame 을 보내지 않는다. local TCP client 를 detach 하고 evidence 를 반환한다. 이후 reconnect, axis/group 재조회와 승인된 safety command 1 회 전송이 필요하다.

`expectedSessionGeneration` overload 는 다른 session 을 잘못 폐기하지 않기 위한 guard 다.

`LMCSendPriorityCoordinator` 는 ordinary/preemptible send 와 priority send 의 순서를 PC process 안에서 조정한다.

이미 wire 에 들어간 RPC 를 물리적으로 취소하지 않으며 PLC E-stop/STO 를 대체하지 않는다.

3. Single Axis API

3.1 LMCSingleAxis 생성

LASAL axis object name 으로 axis reference 를 가져온다.

```
public LMCSingleAxis(
  LMCCConnection connection,
  string axisName)

public static Task<LMCSingleAxis> CreateAsync(
  LMCCConnection connection,
  string axisName,
  CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
connection	LMCCConnection	-	초기화된 RPC connection
axisName	string	ASCII string	PLC 에 등록된 axis object name
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMCSingleAxis	조회된 axis 객체
Task<LMCSingleAxis>	비동기로 조회된 axis 객체

`LMCAxis` 는 `LMCSingleAxis` 의 호환 이름이다.

3.2 PowerOn

Axis Power On 을 요청한다.

```
public LMC_Response PowerOn()
public Task<LMC_Response> PowerOnAsync(
  CancellationToken cancellationToken)

public Task<LMCAxisPowerStateWaitResult>
  PowerOnAndWaitForStableStateAsync(
    CancellationToken cancellationToken)
```

```

public Task<LMCAxisPowerStateWaitResult>
ResumePowerOnWaitForStableStateAsync(
    LMCAxisPowerOnWaitContinuation continuation,
    CancellationToken cancellationToken)

public Task<LMCAxisPowerStateWaitResult> WaitForPowerStateAsync(
    bool expectedPowerOn,
    CancellationToken cancellationToken)

public LMCAxisPowerOnWaitContinuation PendingPowerOnWaitContinuation { get; }
public void ResolvePowerOnWaitAfterStablePowerOff(
    LMCAxisPowerOnWaitContinuation continuation)

```

Parameter	Type	UNIT	설명
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Power On command 결과
Task<LMC_Response>	비동기 Power On command 결과
Task<LMCAxisPowerStateWaitResult>	Power On ACK 와 안정 상태 확인 evidence

PowerOnAndWaitForStableStateAsync 는 0x2023 을 한 번만 보내고 성공 ACK 뒤 0x2028 의 PowerOn=true 를 기본 3 회 연속 확인한다. timeout/cancel 뒤에는 예외나 accepted observer 가 보존한 LMCAxisPowerOnWaitContinuation 을 ResumePowerOnWaitForStableStateAsync 에 전달한다. Resume 과 restart 용 WaitForPowerStateAsync 는 0x2028 만 보내며 Power On 을 다시 보내지 않는다. options overload 로 total deadline, poll interval 과 stable sample count 를 설정할 수 있다. post-write deadline 은 connection 을 Faulted 로 만들 수 있으므로 Evidence.SubmissionOutcome, CommandMayHaveBeenSent, PowerOnAccepted, TransportInvalidatedAtDeadline 을 함께 확인한다. Power On 완료 조건은 read 성공과 PowerOn=true 이며 axis error 가 0 일 필요는 없다. 따라서 성공 결과에서도 FinalStatus.HasAxisError, DS402 상태와 drive error 를 별도로 확인한다.

3.3 PowerOff

Axis Power Off 를 요청한다.

```

public LMC_Response PowerOff()
public Task<LMC_Response> PowerOffAsync(
    CancellationToken cancellationToken)

public Task<LMCAxisPowerOffWaitContinuation>
BeginPowerOffWaitForStableStateAsync(
    CancellationToken cancellationToken)

public Task<LMCAxisPowerOffWaitResult>
ResumePowerOffWaitForStableStateAsync(
    LMCAxisPowerOffWaitContinuation continuation,
    CancellationToken cancellationToken)

public Task<LMCAxisPowerOffWaitResult>
PowerOffAndWaitForStableStateAsync(
    CancellationToken cancellationToken)

public LMCAxisPowerOffWaitContinuation PendingPowerOffWaitContinuation { get; }

```

Return	설명
LMC_Response	Power Off command 결과
Task<LMC_Response>	비동기 Power Off command 결과
Task<LMCAxisPowerOffWaitContinuation>	한 번 accept 된 Power Off 의 status-only 재개 정보
Task<LMCAxisPowerOffWaitResult>	PowerOn=false && Standstill=true 안정 evidence

Begin 은 Power Off 0x2023 을 한 번 보내고 ACK, process-local mutation generation 과 accepted continuation 을 session/send-priority publication 안에서 함께 저장한다. Resume 은 0x2028 만 polling 하고 pre-wire/status publication/final resolution 에서 원 generation 을 확인한다. later same-axis mutation 은 LMCAxisPowerOffInterferenceException 과 expected/observed/intervening evidence 를 반환하고 pending 을 유지하며 PowerOff 를 replay 하지 않는다. final proof 전에 관찰된 cancel/deadline/generation change 는 pending 을 보존하고 proof commit 뒤 late cancel/deadline 은 성공을 뒤집지 않는다. compound API 는 두 단계를 같은 total deadline 으로 조합한다. 외부 PLC/client/direct SDO/group 은 이 귀속 범위 밖이며 성공 ACK 만으로 전원 차단 완료를 판정하지 않는다.

3.4 Reset

Axis error reset 을 요청한다.

```

public LMC_Response Reset()
public Task<LMC_Response> ResetAsync(
    CancellationToken cancellationToken)

public Task<LMCAxisResetWaitResult>
    ResetAndWaitForStableErrorClearanceAsync(
        CancellationToken cancellationToken)

public Task<LMCAxisResetWaitContinuation>
    BeginResetWaitForStableErrorClearanceAsync(
        CancellationToken cancellationToken)

public Task<LMCAxisResetWaitResult>
    ResumeResetWaitForStableErrorClearanceAsync(
        LMCAxisResetWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCAxisStableErrorClearanceWaitResult>
    WaitForStableErrorClearanceAsync(
        CancellationToken cancellationToken)

public LMCAxisResetWaitContinuation PendingResetWaitContinuation { get; }

```

Return	설명
LMC_Response	Reset command 결과
Task<LMC_Response>	비동기 Reset command 결과
Task<LMCAxisResetWaitContinuation>	한 번 accept 된 Reset 의 status-only 재개 정보
Task<LMCAxisResetWaitResult>	Reset ACK 와 native axis error 0 안정 evidence

Begin 은 0x2024 를 한 번만 보내고 accepted continuation 을 반환하며 status 를 읽지 않는다. Resume 은 0x2028 만 보내 AxisErrorId == 0 을 기본 3 회 연속 확인한다. compound API 는 두 단계를 한 total elapsed deadline 으로 조합한다. timeout/cancel/status 실패 뒤에도 continuation 을 status-only 로 재개하고 Reset 을 replay 하지 않는다.

invalid/stale/superseded/completed continuation 과 concurrent second Resume 은 zero-wire 다.

`WaitForStableErrorClearanceAsync` 는 Reset 을 보내지 않는 read-only helper 라서 restart 뒤 상태 재확인에 사용할 수 있다.

같은 connection session 과 `AxisReference` 의 later `LMCSingleAxis` mutation 이 may-have-been-sent boundary 에 도달하면 `LMCAxisResetInterferenceException` 으로 원 Reset 귀속을 거부한다. continuation 은 pending 으로 남지만 의도적인 post-Reset Power On 을 포함해 간섭이 확인된 뒤에는 명시적인 새 Reset 이 필요하다. 외부 PLC, 다른 RPC client, direct SDO 와 group operation 은 이 process-local 귀속 범위 밖이다. 현재 reserved `StatusWord` 를 DS402 Fault 해제 증거로 해석하지 말고, 실제 장비에서는 DS402 Fault, `AxError` 와 drive error register 를 별도로 확인한다.

3.5 Stop

현재 axis motion 의 정지를 요청한다.

```
public LMC_Response Stop(
    int deceleration,
    int jerk)

public Task<LMC_Response> StopAsync(
    int deceleration,
    int jerk,
    CancellationToken cancellationToken)

public Task<LMCAxisStopWaitContinuation>
    BeginStopWaitForStableStandstillAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public Task<LMCAxisStopWaitResult>
    ResumeStopWaitForStableStandstillAsync(
        LMCAxisStopWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCAxisStopWaitResult>
    StopAndWaitForStableStandstillAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public Task<LMCAxisStableStandstillWaitResult>
    WaitForStableStandstillAsync(
        CancellationToken cancellationToken)

public LMCAxisStopWaitContinuation PendingStopWaitContinuation { get; }

public Task<LMCAxisStopWaitContinuation>
    BeginStopWaitForStableStandstillWithResetTakeoverAsync(
        LMCAxisResetWaitContinuation resetContinuation,
        int deceleration,
        int jerk,
        LMCAxisStopWaitOptions options,
        CancellationToken cancellationToken)

public Task<LMCAxisStopWaitResult>
    StopAndWaitForStableStandstillWithResetTakeoverAsync(
        LMCAxisResetWaitContinuation resetContinuation,
        int deceleration,
        int jerk,
        LMCAxisStopWaitOptions options,
        Action<LMCAxisStopWaitContinuation> acceptedContinuationObserver,
        CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
deceleration	int	PLC application UNIT/s² DINT	양수인 정지 감속도. 0/음수는 송신 전에 거부
jerk	int	PLC application UNIT/s³/1000 DINT	0 이상인 정지 jerk. 음수는 송신 전에 거부
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Stop command 결과
Task<LMC_Response>	비동기 Stop command 결과
Task<LMCAxisStopWaitContinuation>	한 번 accept 된 Stop 의 status-only 재개 정보
Task<LMCAxisStopWaitResult>	안정 Standstill 확인 evidence

성공 응답은 LASAL _LMCAxis.StopMove 가 오류 flag 없이 접수됐다는 뜻이다. 실제 정지 완료는

ReadStatusResult[Async]의 IsStandstill 을 후속 확인해야 한다. 이 로컬 DINT 계약을 PMAS/MMCLib MMC_Stop 의 function-block 완료 의미와 동일하게 해석하지 않는다. Begin 은 0x2022 를 정확히 한 번 보내고 continuation 을 반환한다. Resume 은 0x2028 만 polling 하며 기본 3 회 연속 IsSuccess && IsStandstill 을 확인한다. compound API 는 두 단계를 같은 total deadline 으로 조합한다. timeout/cancel/status 실패 뒤 accepted continuation 을 재개할 때 Stop 을 다시 보내지 않는다. 같은 session/AxisReference 의 later LMCSingleAxis mutation 이 확인되면

LMCAxisStopInterferenceException 을 반환하고 pending continuation 을 보존한다. zero-wire mutation 과 다른 AxisReference 는 간섭하지 않으며 외부 PLC/client/SDO/group operation 은 process-local 귀속 범위 밖이다.

WaitForStableStandstillAsync 는 Stop 을 보내지 않고 0x2028 만 읽는다. 이미 안정 Power Off 가 증명된 뒤에는 TryRetirePendingStopAfterStablePowerOff 로 정확한 pending Stop 을 wire traffic 없이 superseded 처리할 수 있다. ...WithResetTakeoverAsync overload 는 정확한 same-session pending Reset 의 소유권을 새 Stop 에 원자적으로 넘기는 고급 recovery API 다. 임의의 stale/foreign Reset continuation 에는 사용할 수 없으며 Stop ACK 가 불명확하면 Reset 이나 Stop 을 blind retry 하지 않는다.

3.6 ReadStatus

Axis 상태를 읽는다.

```
public uint ReadStatus()
public uint ReadStatus(out LMC_Response response)

public LMCReadStatusResult ReadStatusResult()
public Task<LMCReadStatusResult> ReadStatusResultAsync(
    CancellationToken cancellationToken)
```

Return	설명
uint	Raw axis state
LMCReadStatusResult	Axis 상태와 error 정보
Task<LMCReadStatusResult>	비동기 axis 상태와 error 정보

3.7 GetActualPosition

Axis actual position 을 읽는다.

```
public int GetActualPosition()
public int GetActualPosition(out LMC_Response response)

public LMCReadActualPositionResult GetActualPositionResult()
public Task<LMCReadActualPositionResult> GetActualPositionResultAsync(
    CancellationToken cancellationToken)
```

Return	UNIT	설명
int	PLC application UNIT DINT	Raw actual position
LMCReadActualPositionResult	PLC application UNIT DINT	Actual position 과 error 정보
Task<LMCReadActualPositionResult>	PLC application UNIT DINT	비동기 actual position 결과

3.8 MoveAbsoluteEx

Axis 를 absolute position 으로 이동시킨다.

```
public LMC_Response MoveAbsoluteEx(
    int position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction = LMC_DIRECTION.Shortest)

public Task<LMC_Response> MoveAbsoluteExAsync(
    int position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
position	int	PLC application UNIT DINT	Absolute target position
velocity	int	PLC application UNIT/s DINT	Velocity
acceleration	int	PLC application UNIT/s² DINT	Acceleration
deceleration	int	PLC application UNIT/s² DINT	Deceleration
jerk	int	PLC application UNIT/s³/1000	Jerk

Parameter	Type	UNIT	설명
		DINT	
direction	LMC_DIRECTION	Shortest	이동 방향
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveAbsolute command 결과
Task<LMC_Response>	비동기 MoveAbsolute command 결과

3.9 MoveRelativeEx

현재 위치에서 지정한 distance 만큼 이동시킨다.

```
public LMC_Response MoveRelativeEx(
    int distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction = LMC_DIRECTION.Shortest)

public Task<LMC_Response> MoveRelativeExAsync(
    int distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
distance	int	PLC application UNIT DINT	Signed relative distance
velocity	int	PLC application UNIT/s DINT	Velocity
acceleration	int	PLC application UNIT/s² DINT	Acceleration
deceleration	int	PLC application UNIT/s² DINT	Deceleration
jerk	int	PLC application UNIT/s³/1000 DINT	Jerk
direction	LMC_DIRECTION	Shortest	Distance 부호에 따른 이동
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveRelative command 결과
Task<LMC_Response>	비동기 MoveRelative command 결과

3.10 MoveVelocityEx

지정한 방향과 속도로 axis 를 구동한다.

```
public LMC_Response MoveVelocityEx(
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction)

public Task<LMC_Response> MoveVelocityExAsync(
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
velocity	int	PLC application UNIT/s DINT	Velocity magnitude
acceleration	int	PLC application UNIT/s² DINT	Acceleration
deceleration	int	0	0 을 전달하고 정지는 Stop 사용
jerk	int	PLC application UNIT/s³/1000 DINT	Jerk
direction	LMC_DIRECTION	Positive, Negative	이동 방향
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveVelocity command 결과
Task<LMC_Response>	비동기 MoveVelocity command 결과

3.11 Drive operation mode 와 composite status

physical axis/slave 1.4 의 CiA 402 operation mode 와 status 를 D5 SDO Read 로 조회한다.

```
public LMCDriveOperationModeResult GetDriveOperationMode()
public LMCDriveOperationModeResult GetDriveOperationMode(uint timeoutCycles)
public Task<LMCDriveOperationModeResult> GetDriveOperationModeAsync(
    CancellationToken cancellationToken)
public Task<LMCDriveOperationModeResult> GetDriveOperationModeAsync(
    uint timeoutCycles,
    CancellationToken cancellationToken)

public LMCDriveErrorCodeResult GetDriveErrorCode()
public LMCDriveErrorCodeResult GetDriveErrorCode(uint timeoutCycles)
public Task<LMCDriveErrorCodeResult> GetDriveErrorCodeAsync(
    CancellationToken cancellationToken)
public Task<LMCDriveErrorCodeResult> GetDriveErrorCodeAsync(
    uint timeoutCycles,
    CancellationToken cancellationToken)

public LMCDriveStatus ReadDriveStatus()
public LMCDriveStatus ReadDriveStatus(uint timeoutCycles)
public Task<LMCDriveStatus> ReadDriveStatusAsync(
    CancellationToken cancellationToken)
public Task<LMCDriveStatus> ReadDriveStatusAsync(
    uint timeoutCycles,
```

```
CancellationToken cancellationToken)
```

`GetDriveOperationMode` 는 `0x6061:0 Int8/1` 을 읽어 typed `Mode` 와 signed `RawValue` 를 반환한다.

unknown/manufacture-specific 값은 `IsKnownMode=false` 여도 `RawValue` 에 보존된다. `GetDriveErrorCode` 는 `0x603F:0 UInt16/2` 를 읽고 raw drive error code 와 D5 ticket evidence 를 반환한다. 값 0 은 해당 read 시점의 drive error register 가 0 이라는 뜻이며 이전 fault 이력이나 DS402 Warning 부재를 증명하지 않는다.

`ReadDriveStatus` 는 LASAL `ReadStatus`, DS402 `0x6041:0 BitField16/2`, `0x6061:0 Int8/1` 을 순차 실행한다. 같은 EtherCAT cycle 의 atomic snapshot 이 아니므로 `IsAtomicSnapshot` 은 항상 false 다. `AxisStatus`, `Ds402StatusWord`, `OperationModeResult` 와 software/hardware/DS402 limit indication 을 source 별로 확인한다.

`timeoutCycles` 는 각 PLC SDO operation timeout 이며 기본값은 1000 cycles 다. library 의 terminal status poll 간격은 capability 의 `BaseCycleTimeUs` 에서 계산하며 최대 poll 수는 `timeoutCycles+32` 다. `BaseCycleTimeUs=0` 이면 요청 전에 실패한다. terminal 실패는 `LMCSdoReadOperationException`, PC poll 한계는 `LMCSdoReadPollingTimeoutException` 으로 ticket/status 를 보존한다. 제출 뒤 async cancellation 은 ticket 을 포함한 `LMCSdoReadWaitCanceledException` 을 발생시키고 PC wait 만 중단하며, 이미 제출한 PLC ticket 을 자동 cancel 하지 않는다. 이미 진행 중인 status RPC 는 응답을 끝까지 수신한 뒤 취소를 보고하므로 connection 은 유지되고 보존된 ticket 을 다시 조회할 수 있다.

3.12 Move 완료와 restart recovery 경계

`MoveAbsoluteEx`, `MoveRelativeEx`, `MoveVelocityEx` 의 sync/async 메서드는 command ACK 만 반환한다. 현재 SDK 에는 `public Move...AndWait`, `BeginMove...`, `ResumeMove...` continuation API 가 없다. `MoveVelocityEx` 는 명시적 Stop 전까지 목표 완료점 자체가 없다.

위치 이동 완료가 필요하면 application 이 `ReadStatusResult[Async]`와 `GetActualPositionResult[Async]`를 반복 조회해 Standstill, axis error 와 목표 위치 tolerance 를 함께 확인해야 한다. 예제 WPF 의 stable-sample 감시와 journal 은 application-local 정책이지 SDK 의 durable Move completion token 이 아니다.

timeout, disconnect 또는 process restart 로 Move 결과가 불명확하면 같은 Move 를 자동 재전송하지 않는다. 새 connection/object lookup 뒤 현재 status/position 을 읽고, 필요한 경우 승인된 `StopAndWaitForStableStandstillAsync` 또는 Power Off 를 정확히 한 번 수행한 후 다음 동작을 결정한다.

4. Group API

4.1 LMCGroupAxis 생성

LASAL group object name 으로 group reference 를 가져온다.

```
public LMCGroupAxis(
    LMCCConnection connection,
    string groupName)

public static Task<LMCGroupAxis> CreateAsync(
    LMCCConnection connection,
    string groupName,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
connection	LMCConnection	-	초기화된 RPC connection
groupName	string	ASCII string	PLC 에 등록된 group object name
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMCGroupAxis	조회된 group 객체
Task<LMCGroupAxis>	비동기로 조회된 group 객체

LMCGroup 은 LMCGroupAxis 의 호환 이름이다.

Group Power/Enable/Reset/Stop/Move 의 성공 ACK 는 method 호출 접수 결과다. 완료 상태는

GroupReadStatusResult 와 필요 시 GroupReadActualPosition 으로 확인한다. 현재 SetKin/Lock/Move 는 X/Y/Z/U 축 1~4 에만 적용된다. 이것은 9 축 동시 group interpolation API 가 아니다.

4.2 GetGroupMembersInfo

Group 에 연결된 axis member 정보를 읽는다.

```
public LMC_Response GetGroupMembersInfo()
public LMCGroupMembersInfoResult GetGroupMembersInfoResult()
public Task<LMCGroupMembersInfoResult> GetGroupMembersInfoResultAsync(
    CancellationToken cancellationToken)
```

Return	설명
LMC_Response	Member 조회 command 결과
LMCGroupMembersInfoResult	Axis count, reference, device ID 와 axis name
Task<LMCGroupMembersInfoResult>	비동기 member 정보 결과

4.3 GroupPowerOn

Group member axis 의 Power On 을 요청한다.

```
public LMC_Response GroupPowerOn()
public Task<LMC_Response> GroupPowerOnAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitContinuation>
    BeginGroupPowerOnWaitForStableStateAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult>
    ResumeGroupPowerStateWaitForStableStateAsync(
        LMCGroupPowerStateWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult>
```

```

GroupPowerOnAndWaitForStableStateAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult> WaitForPowerStateAsync(
    bool expectedPowerOn,
    CancellationToken cancellationToken)

```

Return	설명
LMC_Response	Group Power On command 결과
Task<LMC_Response>	비동기 Group Power On command 결과
Task<LMCGroupPowerStateWaitContinuation>	한 번 accept 된 Group Power On 의 status-only 재개 정보
Task<LMCGroupPowerStateWaitResult>	안정된 PowerOn=true 확인 evidence

Begin/compound API 는 0x204A 를 한 번 보내고 Resume 은 0x2045 만 polling 한다. 기본 완료 조건은 read 성공과 PowerOn=true 3 회 연속이다. WaitForPowerStateAsync 는 Power command 를 보내지 않는 read-only helper 다. PendingGroupPowerStateWaitContinuation 은 같은 connection/session/group 의 latest unresolved Power command 를 제공한다.

4.4 GroupPowerOff

Group member axis 의 Power Off 를 요청한다.

```

public LMC_Response GroupPowerOff()
public Task<LMC_Response> GroupPowerOffAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitContinuation>
    BeginGroupPowerOffWaitForStableStateAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult>
    GroupPowerOffAndWaitForStableStateAsync(
        CancellationToken cancellationToken)

```

Return	설명
LMC_Response	Group Power Off command 결과
Task<LMC_Response>	비동기 Group Power Off command 결과
Task<LMCGroupPowerStateWaitContinuation>	한 번 accept 된 Group Power Off 의 status-only 재개 정보
Task<LMCGroupPowerStateWaitResult>	안정된 PowerOn=false 확인 evidence

Begin/compound API 는 0x204B 를 한 번 보내고, 공통 ResumeGroupPowerStateWaitForStableStateAsync 는 0x2045 만 polling 한다. 완료 조건은 read 성공과 PowerOn=false 3 회 연속이다. Group Power wait 는 member 별 drive-ready 나 DS402 fault 부재를 증명하지 않으므로 필요한 경우 개별 axis drive status 를 추가 확인한다.

4.5 GroupEnable

Group motion profile 을 lock 한다.


```

public LMC_Response GroupEnable()
public Task<LMC_Response> GroupEnableAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupEnableWaitResult>
    GroupEnableAndWaitForLockedStandbyAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupEnableWaitContinuation>
    BeginGroupEnableAndWaitForLockedStandbyAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupEnableWaitResult>
    ResumeGroupEnableAndWaitForLockedStandbyAsync(
        LMCGroupEnableWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupLockedStandbyWaitResult>
    WaitForLockedStandbyAsync(
        CancellationToken cancellationToken)

public LMCGroupEnableWaitContinuation PendingGroupEnableWaitContinuation { get; }

public bool TryReleasePendingGroupEnableForRetry(
    LMCGroupEnableWaitContinuation continuation)
public bool InvalidatePendingGroupEnableWaitStatusProof()

```

Return	설명
LMC_Response	Profile lock command 결과
Task<LMC_Response>	비동기 profile lock 결과
Task<LMCGroupEnableWaitContinuation>	한 번 accept 된 Group Enable 의 status-only 재개 정보
Task<LMCGroupEnableWaitResult>	accepted ACK 와 안정된 PowerOn + Locked Standby evidence
Task<LMCGroupLockedStandbyWaitResult>	command 없이 안정된 PowerOn + Locked Standby 를 읽은 결과

stable wait 는 mutation/status gate, fresh 0x2047, 모든 0x2045 와 poll delay 에 하나의 total deadline 을 적용한다. write commit 전 취소/deadline 은 **NotAttempted**, zero wire 이며 connection 을 재사용하고 mutation proof 는 변경하지 않는다. actual write commit 의 **onWriteCommitted** 에서만 mutation generation 을 갱신하고 pending proof 를 0 으로 reset 한다. caller cancel 이 write 뒤 발생하면 ACK/status 를 drain 하고 accepted evidence 를 게시한 뒤 typed cancellation 을 반환한다. ACK 무응답 deadline 은 **OutcomeUncertain**, continuation 없음, connection **Faulted** 이고, ACK 수락 뒤 status 무응답은 **Accepted**, exact pending continuation, connection **Faulted** 다. 두 경우 모두

TransportInvalidatedAtDeadline=true 다. rejected ACK 는 **Rejected** 이고 continuation 이 없다. 사용 가능한 동일 session 의 accepted continuation 은 Resume 으로 0x2045 만 poll 하며 0x2047 을 replay 하지 않는다. options overload 로 deadline, poll interval 과 stable sample 수를 지정할 수 있다. 이 완료 판정은 PC fake-RPC 계약이며 실제 PLC profile lock 증거가 아니다. **TryReleasePendingGroupEnableForRetry** 는 같은 group/session 에서 Disabled/Unlocked 또는 PowerOff 를 3 회 안정 확인한 exact continuation 만 wire 없이 해제한다.

InvalidatePendingGroupEnableWaitStatusProof 는 Stop/Power Off reservation 경계에서 누적 proof 만 지우며 pending command 를 완료시키지 않는다.

4.6 GroupDisable

Group motion profile 을 unlock 한다.

```

public LMC_Response GroupDisable()

```

```

public Task<LMC_Response> GroupDisableAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupDisableWaitContinuation>
    BeginGroupDisableWaitForStableDisabledAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupDisableWaitResult>
    ResumeGroupDisableWaitForStableDisabledAsync(
        LMCGroupDisableWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupDisableWaitResult>
    GroupDisableAndWaitForStableDisabledAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupStableDisabledWaitResult>
    WaitForStableDisabledAsync(
        CancellationToken cancellationToken)

public LMCGroupDisableWaitContinuation PendingGroupDisableWaitContinuation { get; }

```

Return	설명
LMC_Response	Profile unlock command 결과
Task<LMC_Response>	비동기 profile unlock 결과
Task<LMCGroupDisableWaitContinuation>	한 번 accept 된 Group Disable 의 status-only 재개 정보
Task<LMCGroupDisableWaitResult>	안정된 powered-on Disabled 확인 evidence
Task<LMCGroupStableDisabledWaitResult>	command 없이 stable Disabled 를 읽은 결과

Begin/compound API 는 0x2048 을 한 번 보내고 Resume 은 0x2045 만 polling 한다. 기본 완료 조건은 read 성공, PowerOn=true, Disabled=true, Standby=false 3 회 연속이다. 따라서 Power Off 상태를 Group Disable 완료로 오인하지 않는다. newer stable Group Power Off 가 완료된 경우 TryRetirePendingGroupDisableAfterStablePowerOff 는 정확한 pending Disable 을 wire 없이 superseded 처리하지만, powered-on Disabled 완료를 주장하지 않는다.

4.7 GroupReset

Group error reset 을 요청한다.

```

public LMC_Response GroupReset()
public Task<LMC_Response> GroupResetAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupResetWaitContinuation>
    BeginGroupResetWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

public Task<LMCGroupResetWaitContinuation>
    BeginGroupResetWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitOptions options,
        Action<LMCGroupResetPreparedEvidence> preparedEvidenceObserver,
        Action<LMCGroupResetWaitContinuation> acceptedContinuationObserver,
        CancellationToken cancellationToken)

public Task<LMCGroupResetWaitContinuation>
    AttachGroupResetDurableRecoveryAsync(
        LMCGroupResetDurableRecoveryRecord record,
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

public Task<LMCGroupResetWaitResult>
    ResumeGroupResetWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitContinuation continuation,
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

```

```

public Task<LMCGroupResetWaitResult>
    GroupResetAndWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

public bool SupersedePendingGroupResetAfterCapturedMemberSafetyMutation(
    LMCGroupResetWaitContinuation continuation,
    LMCSingleAxis memberAxis)

public LMCGroupResetWaitContinuation PendingGroupResetWaitContinuation { get; }

```

Return	설명
LMC_Response	Group Reset command 결과
Task<LMC_Response>	비동기 Group Reset command 결과
Task<LMCGroupResetWaitContinuation>	fresh Reset 의 same-session 또는 exact durable reconnect/restart status-only 재개 정보
Task<LMCGroupResetWaitResult>	pinned group/member error-clear 가 안정된 결과와 evidence

`raw GroupReset[Async]`는 ACK-only 호환 API 다. `stable Begin` 은 성공한 `0x20D2` observed member snapshot 을 고정 한 뒤 `0x2049` 를 정확히 한 번 보내고, `Resume` 은 `Reset` 을 재전송하지 않고 각 round 에 `0x2045` 한 번과 pinned member 별 `0x2028` 을 보낸다. group/member error 가 모두 0 인 full-clear round 를 기본 3 회 연속 확인해야 완료다. generic snapshot validation 은 1..16 개 nonzero/unique axis reference 를 허용하며 expected topology 또는 현재 PLC build 증명이 아니다.

timeout/cancel/status failure 뒤에는 같은 live session 의 continuation 으로 status-only `Resume` 한다. accepted 또는 outcome-uncertain `Stop/PowerOff/safe Disable` 이나 pinned-member mutation 은 원 `Reset` 귀속을 terminal supersede 하며, valid safety NACK 와 pre-wire failure 는 continuation 을 보존한다. WPF 같은 coordinator 가 captured-member Axis `Stop/PowerOff` 결과를 처리할 때는

`SupersedePendingGroupResetAfterCapturedMemberSafetyMutation` 으로 exact continuation/member 와 실제 generation mismatch 를 확인해 SDK pending 을 즉시 정리할 수 있다. valid NACK 처럼 generation 이 rollback 된 경우 false 를 반환하고 pending 을 보존한다.

`preparedEvidenceObserver` 는 valid `0x20D2` snapshot 뒤 실제 `0x2049` write 가 시작되기 직전에 operation ID, old owner session, exact ordered member identity 와 stable count 를 전달한다. caller 는 이 동기 경계에서 command-before durable record 를 먼저 저장해야 한다. observer 가 실패하거나 reentrant mutation 을 시도하면 `0x2049` 는 전송되지 않는다.

process restart/reconnect 에서는 old continuation 을 재사용하지 않는다. endpoint,

`DiagnosticsBuild/BootId/MapRevision` 와 group identity 를 먼저 확인한 뒤 `AttachGroupResetDurableRecoveryAsync` 가 fresh `0x20D2` 를 한 번 읽어 stored member count/order/name/ reference/device 를 exact-match 한다. 일치할 때만 새 session 의 recovery continuation 을 만들며 attach 와 `Resume` 모두 `0x2049` 를 보내지 않는다. stored prior outcome 이 `OutcomeUncertain` 이면 성공 결과도 현재 group/member error 가 안정적으로 clear 됐다는 사실만 뜻하며 이전 `Reset` ACK 나 성공을 추론하지 않는다. WPF 는 이 record 를 checksum, single-writer lock 과 exact CAS 로 보존하고 stable proof 또는 accepted/outcome-uncertain safety supersede 뒤에만 resolve 한다.

이 결과는 LASAL application error-clear 관찰이지 DS402 Fault, drive error register, power, profile lock, Home/reference 또는 motion-ready 증거가 아니다.

4.8 GroupStop

현재 group motion 의 정지를 요청한다.

```
public LMC_Response GroupStop(
    int deceleration,
    int jerk)

public Task<LMC_Response> GroupStopAsync(
    int deceleration,
    int jerk,
    CancellationToken cancellationToken)

public Task<LMCGroupStopWaitContinuation>
    BeginGroupStopWaitForStableStandbyAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public Task<LMCGroupStopWaitResult>
    ResumeGroupStopWaitForStableStandbyAsync(
        LMCGroupStopWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupStopWaitResult>
    GroupStopAndWaitForStableStandbyAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public LMCGroupStopWaitContinuation PendingGroupStopWaitContinuation { get; }
```

Parameter	Type	UNIT	설명
deceleration	int	Group application UNIT/s ² DINT	정지 감속도
jerk	int	Group application UNIT/s ³ /1000 DINT	정지 jerk
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Group Stop command 결과
Task<LMC_Response>	비동기 Group Stop command 결과
Task<LMCGroupStopWaitContinuation>	한 번 accept 된 Group Stop 의 status-only 재개 정보
Task<LMCGroupStopWaitResult>	안정 Standby 확인 evidence

deceleration/jerk 조합은 PLC 계약에 맞게 RPC 전에 검사한다. success ACK 는 입력 검증, robot client 연결과 StopMove(Mode:=3) dispatch 를 뜻하며 정지 완료가 아니다. StopMove() 반환 StopCmdNo 는 오류 코드가 아니라 정지가 끝날 buffer command index 다. 실제 완료와 profile error 는 GroupReadStatusResult 로 확인한다.

Begin 은 0x2085 를 정확히 한 번 보내고 success ACK 를 connection/session/group/latest-pending 에 묶인 continuation 으로 반환하며 status 를 읽지 않는다. Resume 은 그 exact continuation 으로 0x2045 만 poll 하여 IsStandby 를 기본 3 회 연속 확인한다. timeout/cancel/status failure 또는 send-priority preemption 뒤에도 accepted continuation 과 evidence 를 보존하며 owner session 이 사용 가능하면 원 Stop 없이 Resume 할 수 있다.

TransportInvalidatedAtDeadline=true 이면 그 session 의 continuation 은 재사용할 수 없고 reconnect 뒤에도 Stop 을 자동 replay 하지 않는다. PendingGroupStopWaitContinuation 은 현재 handle 과 같은 connection/session/group 의

latest pending Stop 만 노출한다. stale, superseded, completed continuation 과 concurrent second Resume 은 zero-wire 로 거부된다. options overload 로 deadline, poll interval 과 stable sample 수를 지정할 수 있으며 compound API 는 Begin 과 Resume 에 같은 elapsed total deadline 을 적용한다.

4.9 GroupReadStatus

Group power 와 profile 상태를 읽는다.

```
public uint GroupReadStatus()
public uint GroupReadStatus(out LMC_Response response)

public LMCGroupReadStatusResult GroupReadStatusResult()
public Task<LMCGroupReadStatusResult> GroupReadStatusResultAsync(
    CancellationToken cancellationToken)
```

Return	설명
uint	Raw group state
LMCGroupReadStatusResult	Group power, profile 상태와 error 정보
Task<LMCGroupReadStatusResult>	비동기 group 상태 결과

4.10 GroupReadActualPosition

Group actual position 을 읽는다.

```
public LMCGroupReadActualPositionResult GroupReadActualPosition(
    LMC_COORD_SYSTEM coordinateSystem)

public Task<LMCGroupReadActualPositionResult> GroupReadActualPositionAsync(
    LMC_COORD_SYSTEM coordinateSystem,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
coordinateSystem	LMC_COORD_SYSTEM	Enum	None 또는 Acs member-slot alias
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	UNIT	설명
LMCGroupReadActualPositionResult	Group application UNIT DINT	slot 1..9 member position, slot 10..16 zero 와 error 정보
Task<LMCGroupReadActualPositionResult>	Group application UNIT DINT	비동기 group position 결과

현재 adapter 는 no-CalcModel static identity 에서 None/Acs 를 같은 GetRobotPosition(CoordSystem:=0) member-slot read alias 로 처리한다. Mcs/Pcs 는 지원하지 않아 C#에서 NotSupportedException 이 발생하며, 구 SDK 요청은 PLC 가 ErrorId=-7 로 거부한다. slot 1..9 는 software group member 순서이고 slot 10..16 은 0 이다. Acs alias 의 실물 동등성은 PLC 시험이 남아 있다.

4.11 SetKinTransformCartesian4Axis

4 개 axis 를 Cartesian X/Y/Z/U 로 설정한다.

```
public LMC_Response SetKinTransformCartesian4Axis(
    LMCSingleAxis axisX,
    LMCSingleAxis axisY,
    LMCSingleAxis axisZ,
    LMCSingleAxis axisU)

public Task<LMC_Response> SetKinTransformCartesian4AxisAsync(
    LMCSingleAxis axisX,
    LMCSingleAxis axisY,
    LMCSingleAxis axisZ,
    LMCSingleAxis axisU,
    CancellationTokn cancellationToken)
```

Parameter	Type	UNIT	설명
axisX	LMCSingleAxis	-	Cartesian X axis
axisY	LMCSingleAxis	-	Cartesian Y axis
axisZ	LMCSingleAxis	-	Cartesian Z axis
axisU	LMCSingleAxis	-	Cartesian U axis
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Kinematic transform command 결과
Task<LMC_Response>	비동기 transform command 결과

이 helper 는 exact X/Y/Z/U identity payload 만 만든다. generic/dynamic kinematic transform 계산이나 profile lock 을 수행하지 않는다.

4.12 MoveLinearAbsoluteEx

Group 을 Cartesian absolute position 으로 linear 이동시킨다.

```
public LMC_Response MoveLinearAbsoluteEx(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk)

public LMC_Response MoveLinearAbsoluteEx(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMCGroupMotionOptions options)

public Task<LMC_Response> MoveLinearAbsoluteExAsync(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    CancellationTokn cancellationToken)
```

```
public Task<LMC_Response> MoveLinearAbsoluteExAsync(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMCGroupMotionOptions options,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
position	int[]	Group application UNIT DINT	wire 16 개; 현재 PLC 는 X/Y/Z/U slot 1..4 만 사용하고 5..16 은 0 이어야 함
velocity	int	Group application UNIT/s DINT	Path velocity
acceleration	int	Group application UNIT/s ² DINT	Path acceleration
deceleration	int	Group application UNIT/s ² DINT	Path deceleration
jerk	int	Group application UNIT/s ³ /1000 DINT	Path jerk
options	LMCGroupMotionOptions	-	Coordinate, transition, buffer 와 execute 설정
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveLinearAbsolute command 결과
Task<LMC_Response>	비동기 MoveLinearAbsolute command 결과

현재 C#과 PLC 가 함께 허용하는 범위는 position slot 1..4 X/Y/Z/U, 나머지 0, 양수 velocity/acceleration/deceleration, 0 이상 jerk, coordinate **None**, transition **ExactStop/ContinuousDirect**, buffer **Aborting/Buffered**, **Execute=true** 다. 정의됐지만 지원하지 않는 option 은 RPC 전에 **NotSupportedException** 으로 거부한다.

4.13 MoveLinearRelativeEx

Group profile 의 마지막 buffered target 을 기준으로 Cartesian relative distance 를 원자적으로 적재한다. PC 에서 현재 위치를 읽어 absolute target 으로 변환하지 않는다.

```
public LMCAdminResponse MoveLinearRelativeEx(
    int[] distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk)

public LMCAdminResponse MoveLinearRelativeEx(
    int[] distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMCGroupMotionOptions options)

public Task<LMCAdminResponse> MoveLinearRelativeExAsync(
    int[] distance,
```

```
int velocity,
int acceleration,
int deceleration,
int jerk,
LMCGroupMotionOptions options,
CancellationToken cancellationToken)
```

`distance` 와 `dynamics/options` 의 UNIT 및 허용 범위는 absolute move 와 같다. 현재 PLC 는 X/Y/Z/U slot 1..4 만 사용하고 slot 5..16=0, coordinate `None`, transition `ExactStop/ContinuousDirect`, buffer `Aborting/Buffered`, `Execute=true` 만 허용한다.

반환형은 `LMCAdminResponse` 다. valid command rejection 은 `LMCAdminCommandException.Response` 에 Admin detail 과 native error 를 보존한다. success 는 `MoveRelativeCoord` 가 profile queue 에 명령을 수락했다는 뜻이며 완료가 아니다. 이후 `GroupReadStatusResult[Async]`에서 InPosition/profile error 를 확인한다.

일반 overload 는 같은 session 의 `0x7D00` capability 를 먼저 확인한다. Stop/PowerOff 우선순위 gate 가 있는 UI 는 gate 밖에서 `GetCapabilitiesAsync` 를 수행한 뒤 아래 prepared overload 를 gate 안에서 사용한다. 전달한 capability 는 같은 `LMCConnection` 과 session, feature/group reference 가 아니면 wire 송신 전에 거부된다.

```
LMCAdminCapabilities capabilities =
    await connection.Admin.GetCapabilitiesAsync(cancellationToken);

LMCAdminResponse accepted = await group.MoveLinearRelativeExAsync(
    distance,
    velocity,
    acceleration,
    deceleration,
    jerk,
    options,
    capabilities,
    cancellationToken);
```

prepared capability 를 받는 sync overload 도 같은 인자 순서로 제공한다.

4.14 LMCGroupMotionOptions

`MoveLinearAbsolute/Relative` 의 좌표계와 motion mode 를 설정한다.

Property	Type	UNIT / Default	설명
CoordinateSystem	LMC_COORD_SYSTEM	None	Coordinate system
TransitionMode	LMC_GROUP_TRANSITION_MODE	ExactStop	Transition mode
BufferMode	LMC_BUFFER_MODE	Aborting	Buffer mode
Execute	bool	true	Command execute

현재 PLC adapter 가 승인하는 조합은 `CoordinateSystem=None`, `TransitionMode=ExactStop/ContinuousDirect`, `BufferMode=Aborting/Buffered`, `Execute=true` 뿐이다. public enum 에 다른 값이 있어도 현재 PLC 지원을 뜻하지 않는다.

5. Return Type

5.1 LMCReadStatusResult

ReadStatusResult 의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
IsReadSuccessful	bool	-	RPC/function read 성공; native axis error 존재 여부와 분리
State	uint	Bit field	Raw axis state
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
IsPowerOn	bool	-	Axis Power On 상태
IsReferenced	bool	-	Home / Reference 완료 상태
IsStandstill	bool	-	Standstill 상태
AxisErrorId	ushort	Error ID	Axis error
HasAxisError	bool	-	AxisErrorId != 0
AxisErrorFlags	ushort	Bit field	raw LASAL _LMCAXIS_ERROR; DS402 statusword bit 가 아님
StatusWord	ushort	Reserved	현재 LASAL adapter 는 0 을 반환하며 DS402 statusword 로 사용하지 않음
ErrorId	short	Error ID	Command error

5.2 LMCReadActualPositionResult

GetActualPositionResult 의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
PositionRaw	int	PLC application UNIT DINT	Actual position
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error

Property	Type	UNIT	설명
			bit 여부
ErrorId	short	Error ID	Command error

5.3 LMCGroupReadStatusResult

GroupReadStatusResult 의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
IsReadSuccessful	bool	-	RPC/function read 성공; native group error 존재 여부와 분리
State	uint	Bit field	Raw group state
IsPowerOn	bool	-	Group power 상태
IsStandby	bool	-	Profile locked 상태
IsEnabled	bool	-	IsStandby 호환 alias; servo power 와 다름
IsDisabled	bool	-	Profile unlocked 상태
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
GroupErrorId	ushort	Error ID	Group / profile error
HasGroupError	bool	-	GroupErrorId != 0
ErrorId	short	Error ID	Command error

5.4 LMCGroupReadActualPositionResult

GroupReadActualPosition 의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
CoordinateSystem	LMC_COORD_SYSTEM	Enum	요청에 사용한 좌표계의 PC-side echo; PLC 응답 필드가 아님
PositionsRaw	int[16]	Group application UNIT DINT	slot 1..9 software member

Property	Type	UNIT	설명
			position, slot 10..16 zero
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
ErrorId	short	Error ID	Command error

현재 tracked PLC source 는 `_LMC_PROF_POS` 의 Pos1..Pos9 를 response slot 1..9 에 복사하고 slot 10..16 을 0 으로 유지한다.
Move/SetKin/Lock 의 physical 4 축 제한은 그대로다.

5.5 LMCGroupMembersInfoResult

`GetGroupMembersInfoResult` 의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
AxisCount	byte	Count	Group member 수; 현재 tracked source 는 9
Members	LMCGroupMemberInfo[]	-	Member 정보 배열
AxisReferences / DeviceIds / AxisNames	배열	-	방어 복사된 원본 member 배열
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
ErrorId	short	Error ID	Command error

`LMCGroupMemberInfo` 의 반환 필드는 다음과 같다.

Property	Type	UNIT	설명
Index	int	Index	0-based member index
AxisReference	ushort	Reference	Axis reference
DeviceId	ushort	Device ID	PLC device ID
AxisName	string	ASCII string	LASAL axis object name

5.6 LMCAdminResponse

LASAL-local Admin 명령의 공통 16-byte response 를 보존한다.

Property	Type	설명
TransportResponse	LMC_Response	outer transport frame
SchemaVersion / ResponseFlags	ushort	Admin schema 와 reserved flags
CommandStatus	ushort	0 success, 1 domain rejection
ErrorId	short	Admin -31000, positive GroupProfile code 또는 adapter fallback -6
RequestId	uint	request echo
DetailCode / DetailCodeValue	enum / uint	typed/raw Admin detail
IsSuccess	bool	status/error/detail 이 모두 success 인지 여부

6. Admin 과 Diagnostics facade

6.1 Admin capability 와 semantic parameter read

연결된 `LMCConnection.Admin` 에서 LASAL-local read-only API 를 사용한다. 각 parameter read 는 먼저 `0x7D00` capability 와 허용 mask 를 확인한다.

```
LMCAdminCapabilities capabilities = connection.Admin.GetCapabilities();

LMCAxisParameterResult axisParameter = connection.Admin.ReadAxisParameter(
    axis,
    LMCAxisParameterKey.MaxVelocity);

LMCGroupParametersResult groupParameters = connection.Admin.ReadGroupParameters(
    group,
    LMCGroupParameterSelection.All);
```

async 메서드는 `GetCapabilitiesAsync`, `ReadAxisParameterAsync`, `ReadGroupParametersAsync` 이며 `CancellationToken` 을 받는다. 축/group 객체 overload 외에 `ushort` reference overload 도 있다. 다른 connection 또는 reconnect 전 stale 객체는 거부한다.

axis read 제한:

- physical AxisReference 1..4
- key: `SoftwareMinPosition`, `SoftwareMaxPosition`,

`EndPositionToleranceWindow`, `MaxVelocity`, `MaxAcceleration`, `ReferencePosition`

- 한 호출에 한 key, 반환 value type `Int32`

`EndPositionToleranceWindow` 는 profile 의 in-position 상태가 아니라 축 end-position tolerance parameter 다. 결과의 `Unit` 과 `Value` 를 함께 확인한다.

group read 제한:

- GroupReference `0x0100`
- selection: `PathVelocityLimit`, `PathAccelerationLimit`, `JerkTime` 또는 조합

- 최대 3 개; unit 은 각각 application UNIT/s, application UNIT/s2, milliseconds

지원하지 않는 PLC/schema/capability 는 `LMCAdminNotSupportedException` 또는 `NotSupportedException`, valid admin error response 는 `LMCAdminCommandException` 으로 보고하며 `Response` 에 status/error/detail 을 보존한다. 이 API 는 read-only 이고 motion 을 생성하지 않는다.

6.2 PI alias 와 Bulk builder/reader

먼저 `GetSignalCatalog` 로 얻은 같은 boot/map 의 catalog 를 사용한다.

```
LMCSignalValue value = connection.Diagnostics.ReadPI(
    catalog,
    "axis1.actual_position");

LMCPIBulkBuilder builder = connection.Diagnostics.CreatePIBulkBuilder(catalog);
builder.AddEntry("axis1.actual_position");
builder.AddEntry("axis2.actual_position");
LMCPIBulkReader reader = builder.Configure();
LMCBulkSnapshot snapshot = reader.Upload();
LMCSignalValueEntry entry = reader.GetEntry("axis1.actual_position");
reader.Release();
```

builder 는 readable catalog entry, 중복, 최대 32 개와 exact `MapRevision` 을 검사한다. Configure 성공 뒤 builder 는 frozen 된다. `GetEntry/TryGetEntry` 는 마지막 성공 Upload 의 snapshot 을 조회하며 새 PLC read 를 수행하지 않는다. 별도 compatibility wire 는 없고 D1 PI Read 와 D2 Bulk command 를 재사용한다. sync 메서드에는 대응하는 `ConfigureAsync`, `UploadAsync`, `ReadStatusAsync`, `ReleaseAsync` 가 있다.

6.3 Project-local error catalog

```
LMCErrorDescription description;
if (LMCErrorCatalog.TryDescribe(
    LMCErrorDomain.AdapterCommand,
    response.ErrorId,
    out description))
{
    Console.WriteLine(description.Symbol);
    Console.WriteLine(description.Resolution);
}
```

지원 domain 은 `AdapterCommand`, `AdminDetail`, `DiagnosticsDetail`, `GroupProfile` 이다. 같은 숫자라도 domain 마다 의미가 다르므로 domain 을 추측하지 않는다. 반환 객체는 `Description`, `Resolution`, `CatalogVersion`, `SourceVersion` 을 제공한다. unknown domain/value 는 false 를 반환한다. 이 catalog 는 현재 project-local 계약이며 Elmo Maestro Personality 전체 error database 가 아니다.

6.4 Diagnostics D0 capability 와 공통 수명

Diagnostics 는 `connection.Diagnostics` 에서 시작한다. reconnect 뒤에는 capability 를 다시 읽어야 하며, 이전 session 에서 얻은 catalog, bulk configuration, recorder identity, operation ticket 와 topology 객체를 재사용하지 않는다.

```
LMCDiagnosticCapabilities capabilities =
    connection.Diagnostics.GetCapabilities();

Task<LMCDiagnosticCapabilities> pending =
    connection.Diagnostics.GetCapabilitiesAsync(cancellationToken);
```

주요 반환값은 `BootId`, `MapRevision`, `Capabilities`, `BaseCycleTimeUs`, Catalog/Bulk/Recorder 최대 크기와 request/response payload 한도다. `BootId=0` 은 초기 제한 상태이므로 Recorder 나 SDO 같은 장기 작업을 시작하지 않는다. 2026-07-30 gate 변경 전 retained PLC 에서 확인된 bit mask 는 `0x0000613F` 였다. 축 1 SDO Write gate 와 TW[20]/TW[19]가 반영된 current source 의 기대값은 bit 9 와 bit 18/19 가 추가된 `0x000C633F` 지만, 고정값으로 가정하지 말고 Rebuild/Link/download 후 매 connection 에서 실제 반환값과 새 `BootId/MapRevision` 을 사용한다.

기능 영역	현재 판정	증거 경계
Admin <code>0x7D00/10/20/22</code>	happy-path + current IDE build 확인	current PLC download 와 전체 실물 값/UNIT matrix 미완료
Catalog, PI Read	happy-path 확인	4 축 x 6 신호, 총 24 개 read catalog
Bulk Read	4-entry happy-path 확인	24-entry, Partial/recovery 와 soak 미완료
D5 SDO Read	1/2/4-byte happy-path 확인	timeout/cancel/abort/orphan 전체 matrix 미완료
Static Topology	7-entry happy-path 확인	configured schema 이며 runtime health 증거 아님
EtherCAT Health	source-active	fault/stale/soak 실기 적격성 미완료
Recorder Single/Ring/Trigger	source-active	PLC runtime/capture 적격성 미완료
SDO Write	축 1 UI[24] source 제한 승인	PLC download 후 bit 9 및 실기 안전/readback 검증 필요; 축 2~4 는 차단
PI/DO Write, Recorder Double	현재 차단	capability, route 또는 allowlist OFF
Node Health/DI	dormant source 구현	bit 15/16 OFF 이며 PLC download/runtime/actual-hardware proof 미완료

ACK 또는 operation ticket 은 작업 완료가 아니다. operation status 의 terminal state 와 result 를 확인한다.

`CancellationToken` 은 PC 대기를 중단할 뿐 이미 PLC 가 accept 한 작업을 자동 취소하지 않는다. 결과가 불명인 mutation 또는 Release 를 blind retry 하지 않는다.

6.5 Diagnostics D1 Catalog, Health 와 PI Read

```

LMCSignalCatalogInfo info = diagnostics.GetSignalCatalogInfo();
LMCSignalCatalogChunk chunk = diagnostics.GetSignalCatalogChunk(
    expectedMapRevision, startIndex, maxEntries);
LMCSignalCatalog catalog = diagnostics.GetSignalCatalog();

LMCEtherCATHealth health = diagnostics.ReadEtherCATHealth();
LMCSignalValue value = diagnostics.ReadPI(signalId);
LMCSignalValue typed = diagnostics.ReadPI(
    signalId, expectedMapRevision, expectedType);

```

각 메서드에는 `CancellationToken` 을 받는 Async overload 가 있다. `GetSignalCatalog()` 는 Info 와 chunk 를 조합하고 count/CRC/map 을 검사한다. 현재 catalog alias 는 각 axis 에 다음 6 개다.

- `target_position_last_tx`
- `digital_outputs_last_tx`

- `control_word_last_tx`
- `actual_position`
- `digital_inputs`
- `status_word`

`ReadPI(catalog, alias)` facade 는 `catalog/session/map/type` provenance 를 wire 전 검사한다. 재접속 뒤 이전 `catalog` 나 `alias` lookup 결과를 사용하면 거부된다. 값은 raw DINT/application UNIT 이며 library 가 PMAS UNIT 변환을 수행하지 않는다.

`ReadEtherCATHealth` 는 Master/Slave 상태, DS402 StatusWord 와 Axis Error 를 읽지만 drive-ready, DS402 Warning 부재, 물리 배선 또는 실제 I/O 정상 동작을 증명하지 않는다.

6.6 Diagnostics D2 Bulk Read

raw facade 는 다음 순서로 사용한다.

```
LMCBulkConfiguration configuration = diagnostics.ConfigureBulk(signalIds);
LMCBulkStatus status = diagnostics.ReadBulkStatus(configuration);
LMCBulkSnapshot snapshot = diagnostics.ReadBulk(configuration);
diagnostics.ReleaseBulk(configuration);
```

각 메서드에는 Async overload 가 있다. `ConfigureBulk` 뒤 `Active` 를 확인하고, `ReadBulk` 의 한 snapshot 에서 같은 PLC cycle 의 entry 를 읽는다. `SnapshotFlags.Partial` 이면 RPC 성공만으로 완료 처리하지 말고 각 entry 의 `EntryStatus` 와 detail 을 검사한다. `ReleaseBulk` 결과가 불명확하면 새 session 에서 자동 재전송하지 않는다.

일반 사용에는 6.2 의 `LMCPIBulkBuilder/LMCPIBulkReader` 를 권장한다. current 4-entry Pending -> Active -> Snapshot -> Release happy-path 는 실기 확인됐지만, 최대 24-entry 와 offline Partial/recovery/soak 는 미완료다.

6.7 Diagnostics D3/D4 Recorder

대표 lifecycle API 는 다음과 같다. 모두 `session/BootId` 에 귀속되며 sync 메서드에는 해당 Async overload 가 있다.

```
LMCRecorderConfigurationHandle handle =
    diagnostics.ConfigureRecorder(configuration);
LMCRecorderIdentity identity = diagnostics.StartRecorder(handle);

diagnostics.TriggerRecorder(identity);
diagnostics.StopRecorder(identity);
LMCRecorderStatus status = diagnostics.GetRecorderStatus(identity);
LMCRecorderHeader header = diagnostics.GetRecorderHeader(identity);
LMCRecorderChunk chunk = diagnostics.ReadRecorderChunk(chunkRequest);

LMCRecorderData data = await diagnostics.DownloadRecorderAsync(
    identity, progress, cancellationToken);
diagnostics.ReleaseRecorderBuffer(identity);
diagnostics.ReleaseRecorder(identity);
```

기본 순서는 `Configure -> Start -> Status polling -> Ready -> Header/Chunk 또는 Download ->`

`ReleaseBuffer -> Release` 다. `LMCRecorderData.GetRawUInt32/GetRawInt32` 로 sample/channel raw 값을 읽을 수 있다. Single, Ring, Trigger mode 는 source-active 지만 current PLC runtime packet capture 와 reconnect/fault 적격성은 완료되지 않았다.

`ReadRecorderBankInventory`, `ReadRecoverableRecorderBankInventory`, `AdoptRecorder`, `AdoptActiveRecorder` 등 recovery API 도 공개되어 있다. 이들은 exact BootId/configuration/ identity 와 journal proof 를 요구하며 live reconnect/adopt matrix 는 미완료다.

현재 사용 금지: `ConfigureRecoverableDoubleRecorder`, Double-bank inventory/adoption 계약은 공개되어 있지만 Recorder Double capability 와 PLC route gate 가 OFF 이고 실제 bank count 가 1 이다. 현재 target 에서는 `UnsupportedFeature` 대상이다.

6.8 Diagnostics D5 SDO Read 와 operation ticket

```
LMCSdoReadResult inline = diagnostics.ReadSdoInline(request);
LMCOperationTicket ticket = diagnostics.SubmitSdo(request);
LMCOperationStatus status = diagnostics.GetOperationStatus(ticket);
diagnostics.CancelOperation(ticket);
LMCSdoResultChunk chunk = diagnostics.ReadSdoResultChunk(chunkRequest);
```

각 메서드에는 Async overload 가 있다. current SDO Read 범위는 Slave 1..4, data width 1/2/4 byte, timeout 1..60000 PLC cycle 과 general inline read 다. `LMCSdoRequest.CreateRead` 로 request 를 만든다. 1/2/4-byte Read 와 TypeMismatch 뒤 동일 BootId recovery happy-path 는 실기 확인됐다.

`SubmitSdo` 의 ticket 을 받은 뒤 `GetOperationStatus` 로 terminal `Completed` 와 `Success` 를 확인한다.

`CancelOperation` 도 operation 결과를 임의로 성공/실패로 확정하지 않으므로 status 를 다시 읽는다. PC polling timeout/cancellation 은 PLC operation 을 자동 중단하지 않는다.

8/12-byte request/result chunk 계약은 공개되어 있으나 current maximum SDO data size 가 4 이고 Extended SDO capability/`0x7E51` route 가 OFF 라 실행할 수 없다.

6.9 Static Topology 와 Dynamic Node/I/O

```
LMCEtherCATTopologyInfo info = diagnostics.GetEtherCATTopologyInfo();
LMCEtherCATTopologyChunk chunk = diagnostics.GetEtherCATTopologyChunk(...);
LMCEtherCATTopology topology = diagnostics.GetEtherCATTopology();
```

Async overload 도 제공한다. `GetEtherCATTopology()`는 Info 뒤 7 개 chunk 를 조합하고 CRC 를 검증한다. current static inventory 는 TopologyRevision `0x15867EEC`, 7 entry(Slave 5 + Slot Module 2)다. 이것은 프로젝트에 설정된 schema 이며 실제 runtime node health, 물리 배선 순서, dynamic I/O 값 또는 drive-ready 상태를 증명하지 않는다.

다음 public contract 가 존재한다. current LASAL source 는 read-owner 두 command 를 구현했지만 capability 를 광고하지 않으므로 정상 public/WPF 경로에서는 preflight 에서 차단된다. output write 는 handler 와 allowlist 가 모두 없다.

API	Command	현재 상태
<code>ReadEtherCATNodeHealth[Async]</code>	<code>0x7E13</code>	LASAL handler/464-byte snapshot 구현, capability OFF, runtime proof 없음
<code>ReadDigitalIO[Async]</code>	<code>0x7E22</code>	LASAL handler/CREVIS input-output shadow 구현, capability OFF, runtime proof 없음
<code>SubmitDigitalOutputWrite[Async]</code>	<code>0x7E23</code>	capability OFF, RT owner/handler/allowlist

API	Command	현재 상태
		없음

`GetApprovedDigitalOutputWriteReferences()`는 현재 빈 목록이다. request DTO 를 생성할 수 있다는 사실은 PLC 실행 지원을 뜻하지 않는다.

6.10 Mutation API 정책

Public API 계약	현재 차단 근거	판정
<code>SubmitPIWrite[Async]</code>	PI Write capability OFF, SDK/PLC allowlist 비어 있음, <code>0x7E21</code> unsupported	실행 금지
<code>SDO CreateWrite + SubmitSdo[Async]</code>	축 1, <code>0x2F00:24</code> , Int32, 4-byte 만 SDK/PLC source 승인	조건부 제한 허용
<code>SubmitDigitalOutputWrite[Async]</code>	DO capability/route/owner/allowlist 없음	실행 금지
Recoverable Double Recorder	Double capability/route gate OFF, single bank	실행 금지

`GetApprovedSdoWriteTargets()`는 현재 축 1 의 Gold UI[24] `0x2F00:24`, Int32/4-byte target 한 건만 반환한다. 축 2~4, 다른 index/subindex/type/length/value range 및 DS402 핵심 객체 `0x6040`, `0x607A`, `0x60FF`, `0x6071` 은 SDK 에서 wire 전에 차단된다.

규범적으로 유일한 SDO Write target 은 Axis 1 `0x2F00:24`(UI[24]) Int32/4-byte 다. Axis 2 through 4 와 그 밖의 모든 target 은 blocked 상태다. Manual SDO Write 는 current-session identity-pinned 절차이며 `DiagnosticsBuild`, `BootId`, `MapRevision` 과 exact target 을 고정한 four-ticket same-value proof 가 모두 PASS 해야 한다.

`EvaluateSdoWritePolicy()`가 ready 가 되려면 새 LASAL build 를 PLC 에 download 한 뒤 fresh capability 의 `SDOWrite` bit 9, `SDORead`, `SDOReadGeneralInline`, nonzero `BootId/MapRevision` 과 payload 한도가 모두 일치해야 한다. 기존 PLC 가 bit 9 를 광고하지 않으면 source 승인 target 이 있어도 제출할 수 없다.

실제 Write 전에 축 1 drive program 에서 UI[24]가 미사용임을 확인하고 PowerOff/Standstill, position 안정, 작업자 승인과 mutation journal 을 모두 통과해야 한다. 최초 실기 시험은 baseline Read, 값 불변 pre-Write guard, byte-identical same-value Write, exact readback 의 서로 다른 4 개 ticket 으로 제한한다. 이 4-ticket qualification 이 PASS 해야 일반 수동 Write 버튼이 열린다. 활성 proof 는 exact `LMCConnection` 인스턴스, session generation, `DiagnosticsBuild`, `BootId`, `MapRevision` 과 승인 target 전체 tuple 에 묶이며 reconnect 나 PLC identity 변경 시 폐기된다. identity mismatch 나 disconnect 를 한 번 관측하면 영구 폐기되며, 실제 second-click 은 SDK 가 mutation gate 안에서 fresh `Build/BootId/MapRevision` 을 proof 와 다시 비교한 뒤에만 `0x7E50` 을 전송한다. mismatch 는 `NotAttempted` 이며 Write frame 은 0 회다. 결과가 불명확한 Write 는 자동 재전송하지 않는다. 현재 source/build 계약은 활성화됐지만 PLC download, UI[24] 소유권 및 실제 EtherCAT mailbox mutation 은 미검증이다.

6.11 Request/result 와 provenance 확인

새 Diagnostics API 는 request, options, continuation, ticket, identity 와 result DTO 를 분리한다. 다음 원칙을 공통 적용한다.

확인 항목	사용 규칙
IsSuccess, typed state/outcome	outer frame 성공과 domain operation 완료를 모두 확인
BootId, MapRevision, session generation	현재 connection 에서 얻은 객체만 사용
BelongsTo / BelongsToCurrentSession	catalog, ticket, topology, I/O/result provenance 검증
RequestId, operation/configuration/recorder ID	ACK, status, chunk 와 release 대상의 identity 일치 확인
Detail, ErrorId, failure context TryGet	예외에서 typed evidence 를 보존하고 숫자만 추측하지 않음
Async CancellationToken	PC wait 취소이며 PLC mutation 취소 보장 아님

이 설명서는 principal facade 와 안전 경계를 설명한다. 모든 DTO property, overload 와 exception 의 완전한 선언 목록은 배포 DLL 의 IntelliSense/XML documentation 과 current source 를 함께 확인한다.

6.12 LMC Home, DS402 Home 과 Encoder Maintenance

LMC Home CurrentPositionZero

LMCSingleAxis 의 current LMC_Home 은 switch-search reference 가 아니다. 실행 전에 읽은 actual position 을 ExpectedActualPosition stale-read guard 로 사용하고 target position 은 0 으로 고정한다. axis motion 을 enable 하거나 Home/limit switch 를 찾지 않는다.

단계	Public API	Command
Prepare	PrepareLMC_Home	wire 없음
Start once	LMC_Home, LMC_HomeAsync	0x7D13
Exact outcome query	ReadLMC_HomeOutcome, ReadLMC_HomeOutcomeAsync	0x7D18
Exact terminal retirement	RetireLMC_HomeOutcome, RetireLMC_HomeOutcomeAsync	0x7D19

```

LMCPreparedHome prepared = axis.PrepareLMC_Home(
    expectedActualPosition,
    timeoutMilliseconds,
    adminCapabilities,
    diagnosticCapabilities,
    LMCHomeExecuteToken.Create());

LMCHomeStartAcknowledgement accepted = axis.LMC_Home(prepared);
LMCHomeOutcomeResult outcome = axis.ReadLMC_HomeOutcome(
    accepted.RecoveryKey,
    adminCapabilities,
    diagnosticCapabilities);

if (outcome.IsTerminal)
{
    axis.RetireLMC_HomeOutcome(
        outcome,
        adminCapabilities,
        diagnosticCapabilities);
}

```

Start ACK 는 완료 증거가 아니며 0x7D13 을 replay 하지 않는다. terminal outcome 과 matching retirement 가 필요하다. Admin feature bit 4 는 current source 에서 ON 이고 WPF 는 LMC Home outcome: 로그에 record state, success, original

status/error/detail, axis status/error, raw/application/internal position set, native/evidence/stop/runtime/generation 을 기록한다. 성공 raw feedback 은 wrap-safe $-2/-1/0/+1/+2$ count 창으로 제한하고 $+/-3$ count 이상은 fail-closed 한다. raw before/after 는 물리 feedback 증거이지 bit-identical sample 계약이 아니다. Axis2 의 8382700 -> 8382701 과 Axis1 의 8027834 -> 8027836 을 이전 raw gate 가 -7 로 오판한 뒤 이 창을 PLC/SDK 에 동기화했다. current 수정의 C78 Rebuild/Download, 새 BootId 와 한 축 단독 runtime proof 는 아직 남아 있다.

DS402 Home method 37

별도 LMC_HomeDS402 는 method 37, Home offset 0, velocity/acceleration/distance/torque 0 의 non-moving current-position-zero 계약이다.

단계	Public API	Command
Prepare	PrepareLMC_HomeDS402 또는 PrepareDs402Home	wire 없음
Start once	LMC_HomeDS402[Async] 또는 Ds402Home[Async]	0x7D15
Exact outcome query	ReadDs402HomeOutcome[Async]	0x7D16
Exact terminal retirement	RetireDs402HomeOutcome[Async]	0x7D17

current source 에는 protocol/state-machine/API 가 있지만 LMC_DIAG_DS402_HOME_ENABLED=FALSE 이고 Admin feature bit 6 도 OFF 다. 따라서 current PLC 의 지원 API 로 실행하지 않는다.

TW[20] / TW[19] Encoder Maintenance

이 경로는 일반 SDO Write 와 분리된 파괴적 유지보수 API 다. 허용 payload 는 TW[20] 0x20FC:0x02 <- UInt16 1, TW[19] 0x20FC:0x01 <- UInt16 1 뿐이다.

단계	Public API	Command
Prepare TW[20]	PrepareTw20EncoderErrorWarningReset	wire 없음
Prepare TW[19]	PrepareTw19MultiturnPositionReset	wire 없음
Start once	StartEncoderMaintenance[Async]	0x7E53
Exact outcome query	ReadEncoderMaintenanceOutcome[Async]	0x7E54
Exact terminal retirement	RetireEncoderMaintenanceOutcome[Async]	0x7E55

각 Prepare 는 대응 request 와 one-shot execute token 을 받는다: LMCtw20EncoderErrorWarningResetRequest + LMCtw20EncoderErrorWarningResetExecuteToken.Create(), 또는 LMCtw19MultiturnPositionResetRequest + LMCtw19MultiturnPositionResetExecuteToken.Create(). current source 는 diagnostics capability bit 18/19 를 ON 으로 광고한다. 그러나 ACK 나 terminal outcome 은 정확한 drive error/warning reset 또는 multi-turn position 변화의 물리 증거가 아니므로 선택 축에서 별도 확인한다.